
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Sintaxis y Semántica de los Lenguajes

TRABAJO PRÁCTICO 1
AUTÓMATAS

Docente: Ing. Roxana Leituz

Curso: K2002

Integrantes:

- Ignacio Masaragian
- Guadalupe Gallini
- Matias Torres
- Kevin Castillo

EJERCICIO 1 - Reconocimiento de constantes enteras

Metodología:

Se recorre la cadena ingresada y se separan los distintos numeros utilizando el caracter @ como separador.

Cada numero obtenido se analiza mediante un automata finito determinista, que permite determinar si corresponde a una constante decimal, octal o hexadecimal.

Los numeros decimales pueden comenzar con signo + o -.

Los octales comienzan con 0 y contienen solamente digitos entre 0 y 7.

Los hexadecimales comienzan con 0x o 0X y luego contienen digitos del 0 al 9 o letras entre A-F / a-f.

Si durante el recorrido aparece un caracter que no corresponde al estado actual, el numero se considera un error lexico.

Finalmente se contabiliza la cantidad de numeros decimales, octales y hexadecimales reconocidos.

Definicion formal del automata:

El automata utilizado para reconocer cada numero se define como:

$$A = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{$$

$q_0,$

$q_{\text{Signo}},$

$q_{\text{Decimal}},$

$q_{\text{Cero}},$

$q_{\text{Octal}},$

```
qPrefijoHexadecimal,  
qHexadecimal,  
qError  
}
```

Donde:

q0: estado inicial.

qSigno: se leyo un signo + o -.

qDecimal: se esta reconociendo una constante decimal.

qCero: se leyo un 0 inicial.

qOctal: se esta reconociendo una constante octal.

qPrefijoHexadecimal: se leyo el prefijo 0x o 0X.

qHexadecimal: se esta reconociendo una constante hexadecimal.

qError: se detecto un error lexico.

Alfabeto:

$\Sigma = \{0,1,2,3,4,5,6,7,8,9,+,-,x,X,A,B,C,D,E,F,a,b,c,d,e,f\}$

Estado inicial:

q0

Estados finales:

$F = \{qDecimal, qCero, qOctal, qHexadecimal\}$

Los estados finales permiten ademas identificar el tipo de constante:

Decimal: qDecimal

Octal: qCero o qOctal

Hexadecimal: qHexadecimal

La funcion de transicion δ se implementa mediante los cambios de estado realizados en la funcion analizar_numero().

Cualquier transicion no valida conduce al estado qError.

Estado actual	+ / -	0	1-7	8-9	x / X	A-F / a-f	Otro
q0	qSigno	qCero	qDecimal	qDecimal	qError	qError	qError
qSigno	qError	qDecimal	qDecimal	qDecimal	qError	qError	qError
qDecimal	qError	qDecimal	qDecimal	qDecimal	qError	qError	qError
qCero	qError	qOctal	qOctal	qError	qPrefijoHexadecimal	qError	qError
qOctal	qError	qOctal	qOctal	qError	qError	qError	qError
qPrefijoHexadecimal	qError	qHexadecimal	qHexadecimal	qHexadecimal	qError	qHexadecimal	qError
qHexadecimal	qError	qHexadecimal	qHexadecimal	qHexadecimal	qError	qHexadecimal	qError
qError	qError	qError	qError	qError	qError	qError	qError

Estados de aceptacion:

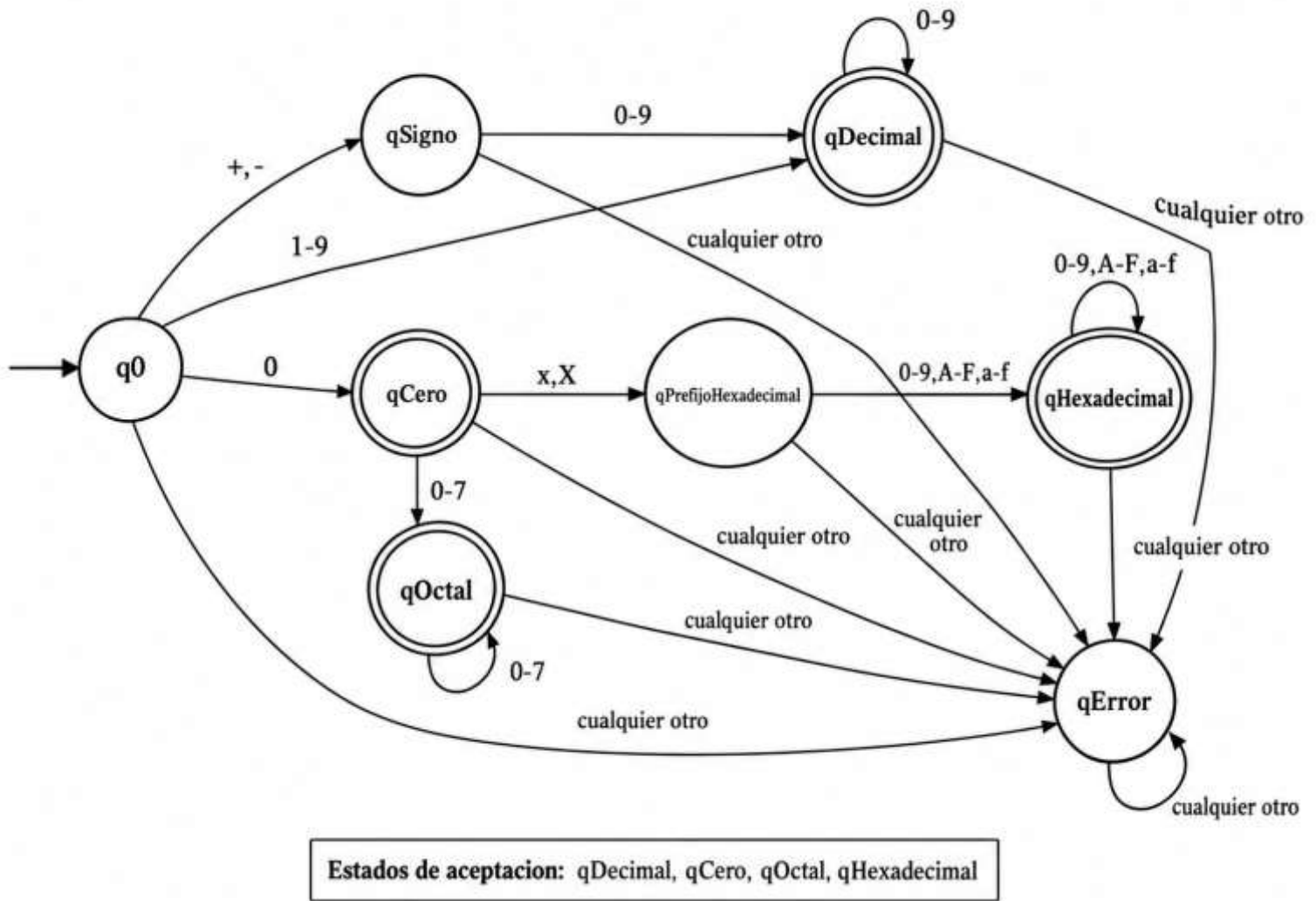
qDecimal -> constante decimal.

qCero -> constante octal formada solamente por 0.

qOctal -> constante octal.

qHexadecimal -> constante hexadecimal.

AFD - Ejercicio 1: reconocimiento de constantes enteras



Autómata finito determinista del Ejercicio 1. El autómata reconoce constantes enteras decimales, octales y hexadecimales, detectando errores léxicos cuando la cadena no pertenece al lenguaje esperado.

EJERCICIO 2 - Conversion de caracter numerico a entero

Metodologia:

Se recibe un caracter numerico comprendido entre '0' y '9' y se devuelve su valor entero correspondiente.

La conversion se realiza restando el caracter '0' al caracter recibido:

$\text{valor} = c - '0'$

Esto permite obtener directamente el valor numerico representado por el caracter.

Por ejemplo:

$'7' - '0' = 7$

código del ejercicio:

```
int caracterAEntero(char c){  
    return c - '0';  
}
```

EJERCICIO 3 - Validacion y evaluacion de expresiones

Metodologia:

Se recibe una expresion formada por numeros enteros decimales y los operadores +, - y *.

Antes de realizar la operacion, la expresion se valida mediante un automata finito determinista.

El automata controla que:

- La expresion comience con un numero.
- Un numero pueda estar formado por uno o mas digitos.
- Luego de un numero pueda aparecer un operador.
- Luego de un operador obligatoriamente deba aparecer otro numero.
- La expresion termine en un numero.

Si la expresion no cumple estas condiciones, se informa un error lexico.

Si la expresion es valida, se calcula el resultado respetando la prioridad de la multiplicacion sobre la suma y la resta.

Definicion formal del automata:

$$A = (Q, \Sigma, \delta, q_0, F)$$

$Q = \{$
 $q_0,$
 $q_{\text{Numero}},$
 $q_{\text{Operador}},$
 q_{Error}
 $\}$

Donde:

q0: estado inicial, espera el comienzo de un numero.

qNumero: se esta leyendo un numero.

qOperador: se leyo un operador y se espera el siguiente numero.

qError: se detecto una expresion invalida.

Alfabeto:

$\Sigma = \{0,1,2,3,4,5,6,7,8,9,+,-,*\}$

Estado inicial:

q0

Estados finales:

F = {qNumero}

Tabla de transiciones del Ejercicio 3

Estado actual	0-9	+ / - / *	Otro
q0	qNumero	qError	qError
qNumero	qNumero	qOperador	qError
qOperador	qNumero	qError	qError
qError	qError	qError	qError

Estado de aceptacion:

qNumero

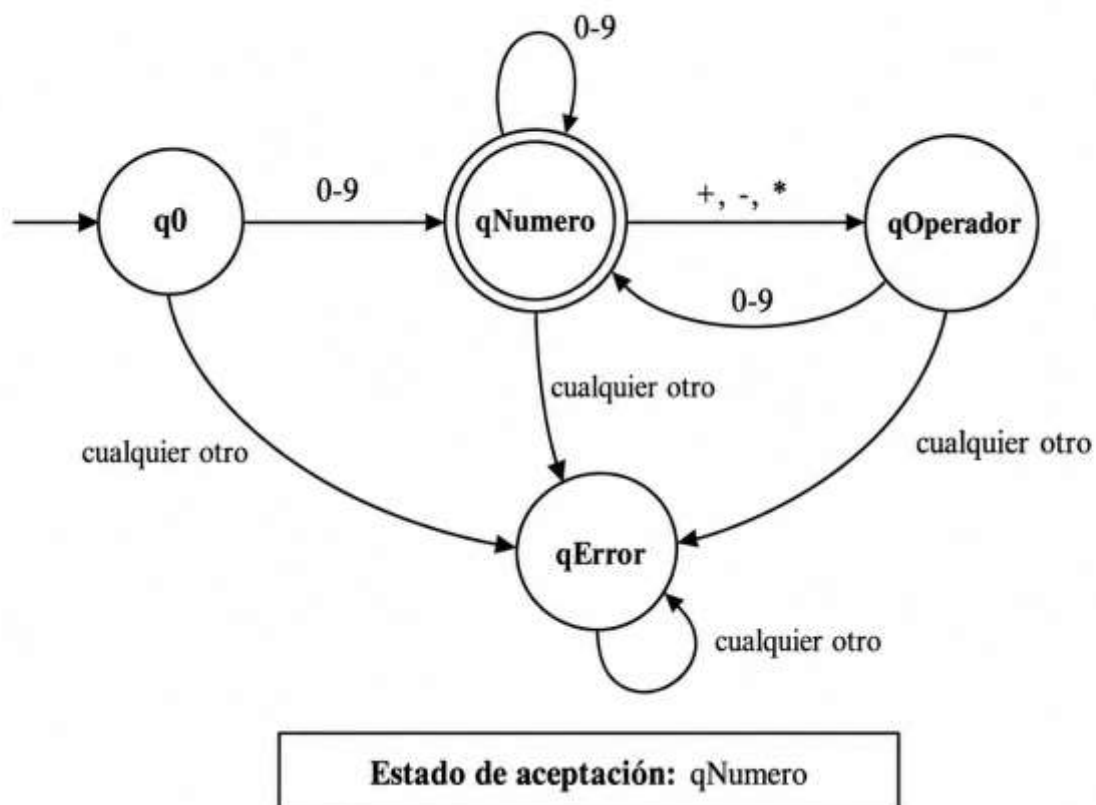
Interpretacion:

- q0: estado inicial, espera el primer dígito de un número.
- qNumero: se está leyendo un número; pueden seguir más dígitos o aparecer un operador.
- qOperador: se leyó un operador y obligatoriamente debe venir otro número.
- qError: la expresión no pertenece al lenguaje esperado.

Automata finito determinista del Ejercicio 3:

El autómata del Ejercicio 3 valida expresiones formadas por enteros decimales y los operadores +, - y *. La expresión debe comenzar con un número, alternar correctamente entre números y operadores, y finalizar en un número. Si no cumple estas condiciones, se considera un error léxico.

AFD - Ejercicio 3: validación de expresiones



INSTRUCTIVO DE COMPILACIÓN Y EJECUCIÓN

El programa fue desarrollado en lenguaje C.

Para compilarlo desde una terminal con GCC:

```
gcc Tp2_SSL.c -o Tp2_SSL.exe
```

Para ejecutarlo en Windows:

```
.\Tp2_SSL.exe
```

Durante la ejecución el programa solicita, en orden:

1. Una cadena de constantes enteras separadas por @.
2. Un carácter numérico.
3. Una expresión aritmética formada por números decimales y los operadores +, - y *.

Luego se muestran por pantalla los resultados obtenidos y, cuando corresponde, los errores léxicos detectados.

Archivos de la entrega

- Tp2_SSL.c: código fuente del programa.
- TP AUTOMATAS 2026 SSL.pdf: documentación, autómatas y pruebas.

Pruebas:

TP Automatas Prueba General:

```
PS C:\Users\Usuario\SSL> .\Tp2_SSL.exe
Ingrese una cadena: 123@0@077@0xAF@+45@-8
Decimales: 3
Octales: 2
Hexadecimales: 1
Ingrese un caracter numerico: 7
Numero entero: 7
Ingrese una expresion: 3+4*7+3-5
Expresion valida
Resultado: 29
PS C:\Users\Usuario\SSL> █
```

TP Automatas Errores:

```
PS C:\Users\Usuario\SSL> .\Tp2_SSL.exe
Ingrese una cadena: 078@0xG@25
Error lexico en 078
Error lexico en 0xG
Decimales: 1
Octales: 0
Hexadecimales: 0
Ingrese un caracter numerico: 7
Numero entero: 7
Ingrese una expresion: 3++4
Error lexico en la expresion
PS C:\Users\Usuario\SSL> █
```